

INSTITUTO FEDERAL DO ESPÍRITO SANTO — CAMPUS CACHOEIRO DE ITAPEMIRIM
Cursos de Sistemas de Informação e Técnico em Informática

Trabalho Interdisciplinar

Estrutura de Dados (C) × Sistemas Operacionais

Edital geral, regras de avaliação e catálogo de 15 temas baseados em árvores

Disciplinas integradas — 15 grupos, um tema único por grupo

Sumário

TOC \h \o "1-2"

Parte I — Edital Geral

1. Visão geral

Cada grupo deverá projetar e implementar, em C, um sistema de média a alta complexidade que integre as disciplinas de Estrutura de Dados e Sistemas Operacionais. O projeto é deliberadamente desenhado para que nenhum assistente de IA o resolva em uma única interação: a integração entre estruturas de dados não triviais e mecanismos reais de sistemas operacionais exige depuração iterativa, decisões de projeto documentadas e análise experimental com dados próprios.

São **15 temas, um por grupo**. Todos têm uma **árvore como estrutura de dados central**, mas cada tema emprega uma família de árvore diferente (de busca em disco, balanceada em memória, trie, de hash, de codificação ou de partição espacial) em um papel diferente, combinada a um mecanismo de Sistemas Operacionais distinto. Assim, o reaproveitamento de código entre grupos é facilmente detectável. A especificação de cada tema está na **Parte III** deste documento.

2. Arcabouço técnico comum

Independentemente do tema, todos os grupos estão sujeitos às mesmas exigências de engenharia:

- **Linguagem:** C (padrão C11), compilado com gcc -Wall -Wextra -Werror, sem warnings.
- **Persistência real:** quando o tema envolver dados em disco, é proibido manter a estrutura inteira (índice, bitmap, tabela) em memória como fonte da verdade. A RAM serve apenas como cache; a correção é verificada instrumentando o número de leituras/escritas por operação.
- **Concorrência:** quando o tema envolver múltiplas threads/processos, a solução deve ser livre de data races (comprovado com ThreadSanitizer) e de vazamentos (Valgrind/AddressSanitizer). Um único mutex global só é aceito até a Entrega 2.
- **Recuperação a falhas:** temas com persistência transacional devem sobreviver à morte abrupta do processo (kill -9) sem corromper suas estruturas, recuperando-se na reinicialização.
- **Build e testes:** Makefile com alvos all, test, stress e clean, bateria de testes própria e scripts de geração dos gráficos do relatório.

3. Política de uso de Inteligência Artificial

O uso de assistentes de IA (Claude, ChatGPT, Copilot e similares) é **permitido e esperado**, sob as seguintes condições:

1. **Histórico de commits obrigatório:** repositório Git com commits incrementais e mensagens significativas desde a primeira semana. Entregas com poucos commits massivos ("initial commit" com milhares de linhas) serão tratadas como indício de não autoria e levarão a arguição aprofundada.
2. **Diário de engenharia (DIARIO.md):** registro semanal das decisões de projeto, dos bugs encontrados e, quando a IA for usada, qual foi o prompt, o que a IA errou e o que a equipe teve que corrigir.

3. **Defesa é soberana:** a nota final de cada aluno é condicionada à capacidade de explicar e modificar o código ao vivo. Não saber explicar o mecanismo central do próprio tema implica nota zero no critério correspondente, independentemente de o código funcionar.

4. Análise experimental obrigatória

O relatório técnico (8 a 15 páginas) deve apresentar, com gráficos gerados a partir de dados reais coletados pela própria equipe:

- Dedução teórica dos parâmetros centrais da estrutura de dados do tema (ex.: ordem da árvore em função do tamanho do bloco) e confronto com a prática.
- Custo das operações principais (I/O, comparações, tempo) sob cargas de tamanho crescente, comparado à previsão de complexidade assintótica.
- Quando aplicável: comparação entre políticas/algoritmos alternativos (ex.: políticas de cache, algoritmos de substituição, políticas de escalonamento).
- Quando aplicável: escalabilidade da concorrência (vazão com 1, 2, 4, 8 e 16 threads) e identificação do gargalo.
- Quando aplicável: overhead e tempo de recuperação dos mecanismos de consistência (journaling, WAL, snapshots).

5. Cronograma e entregas

O projeto é dividido em 4 entregas incrementais ao longo de 2 semanas, cada uma com defesa oral individual. O escopo abaixo é o esqueleto comum; o anexo de cada tema detalha o conteúdo específico de cada marco.

Entrega	Escopo mínimo (esqueleto comum)	Peso
E1 — Fundação	Estruturas e camada de armazenamento/estado de base, com testes unitários da fundação.	15%
E2 — Núcleo de ED	Estrutura de dados central operacional (busca, inserção, remoção); concorrência com mutex global aceita.	40%
E3 — Núcleo de SO	Mecanismo de SO central (cache, concorrência fina, paralelismo) e operações de dados completas, sem races no TSan.	10%
E4 — Robustez e relatório	Consistência/recuperação, teste de fogo do tema passando, relatório experimental e defesa oral.	35%

Defesa oral individual: a cada entrega, qualquer integrante deve ser capaz de explicar e modificar ao vivo qualquer parte do código (por exemplo, alterar um critério interno e mostrar o efeito). A nota individual pode divergir da nota da equipe.

6. Critérios de avaliação

Critério	Peso
Corretude funcional (bateria de testes + teste de fogo do tema)	15%
Qualidade da estrutura de dados central (correção, serialização, eficiência de I/O)	15%
Mecanismos de SO (cache, concorrência sem races,	10%

Critério	Peso
consistência/recuperação)	
Relatório experimental (rigor, gráficos, confronto teoria × prática)	5%
Defesa oral individual	50%
Engenharia (Makefile, testes automatizados, organização, histórico de commits)	5%

Penalidades automáticas: warnings de compilação (−5%), vazamento de memória no caminho feliz (−10%), data race reportada pelo TSan (−15%), corrupção detectada após falha/recuperação (−20%).

7. Entrega final — pacote

Repositório contendo: código-fonte, Makefile (all, test, stress, clean), bateria de testes própria, scripts de geração dos gráficos, DIARIO.md, relatório em PDF e um vídeo de até 5 minutos demonstrando o teste de fogo do tema seguido de execução/recuperação bem-sucedida.

8. Apresentação final do trabalho

- 02/07/2026
- O grupo deve apresentar o trabalho, mostrando segurança no tema estudado.
- Reforço que os avaliadores (professores) foram devidamente alfabetizados, não havendo assim, a necessidade de leitura dos textos dos slides.
- Momento de Perguntas & Respostas:
 - Os alunos devem defender o tema e apresentar sólidos conhecimentos diante dos questionamentos dos avaliadores e da audiência.

9. Material de apoio

- Tanenbaum & Bos, Modern Operating Systems — sistemas de arquivos, escalonamento, deadlocks, memória.
- Arpaci-Dusseau, Operating Systems: Three Easy Pieces (gratuito online) — implementação de sistemas de arquivos, crash consistency, locking, paginação.
- Cormen et al., Introduction to Algorithms — árvores B, árvores balanceadas, hashing, heaps.
- Código-fonte de referência de sistemas reais (ext2/ext3, SQLite, LevelDB, pigz) conforme o tema.

Parte II — Quadro-resumo dos 15 temas

Tabela para sorteio ou escolha em sala. Todos os 15 temas têm uma árvore como estrutura de dados central; cada tema usa uma família de árvore diferente, em um papel diferente, combinada a um mecanismo de Sistemas Operacionais distinto.

#	Tema	Árvore central (ED)	Mecanismo de SO
1	MiniFS	Árvore B+ em disco	Journaling, cache, locks finos
2	Índice de BD	Árvore B (páginas em disco)	Buffer pool, pin/unpin, fsync
3	Memória virtual	Trie de tradução (page walk)	Page fault, substituição, TLB, swap
4	Escalonador CFS	Árvore rubro-negra por vruntime	Preempção, fatia de tempo, justiça
5	KV store LSM	LSM-Tree (níveis de SSTables)	WAL, compactação em background
6	Roteador IP (LPM)	Trie de Patricia (radix)	Plano de dados concorrente, RCU
7	Deduplicador	Árvore de Merkle por arquivo	Thread pool, mmap vs read
8	Backup snapshots	Árvore persistente (COW)	Atomicidade, fsync, coleta
9	Escalonador de disco	Árvore balanceada de cilindros	Simulação de I/O, aging
10	Message broker	Trie de tópicos (curingas)	Sockets, offsets, durabilidade
11	Compressor	Árvore de Huffman (via heap)	Pipeline de threads, reordenação
12	Page cache	Árvore splay (autoajustável)	Cache de blocos, write-back
13	Colisões N-corpos	Quadtree (partição espacial)	Simulação paralela, barreiras
14	Integridade de disco	Árvore de hash (dm-verity)	Leitura verificada, cache de nós
15	Garbage collector	Árvore de intervalos de memória	mmap, mprotect, varredura de pilha

Famílias de árvore (anti-cópia entre grupos): os 15 temas cobrem famílias distintas — árvores de busca em disco (B, B+), árvores balanceadas em memória (rubro-negra, AVL, splay, intervalos, persistente/COW), tries (radix de tradução, Patricia, de tópicos), árvores de hash (Merkle, dm-verity), árvore de codificação (Huffman), árvore de partição espacial (quadtree) e a estrutura hierárquica de níveis da LSM-Tree. Como cada tema usa uma árvore diferente em um papel diferente, o reaproveitamento de código entre grupos é facilmente detectável.

Calibragem: os temas 1, 5 e 8 são os mais pesados em persistência/consistência; os temas 6, 12 e 13 são os mais pesados em concorrência; os temas 3, 4 e 9 são simuladores (menor carga de implementação, maior carga de análise experimental — nesses, o rigor do relatório deve pesar mais na correção).

Parte III — Especificação dos temas

Cada tema abaixo herda integralmente o Edital Geral (Parte I). O texto de cada anexo especifica a árvore utilizada, a estrutura de dados central, o mecanismo de SO central, a descrição do sistema e o teste de fogo que impede solução trivial.

Tema 1 — MiniFS — Sistema de Arquivos com Árvore B+ em Disco

Árvore utilizada: Árvore B+ persistente.

Estrutura de dados (núcleo de ED): Árvore B+ em que cada nó é mapeado para exatamente um bloco do arquivo de disco; chaves são caminhos absolutos, folhas encadeadas para range scan, splits/merges serializados em disco.

Mecanismo de SO (núcleo de SO): Buffer cache LRU, journaling com write-ahead log, locks de granularidade fina (lock coupling na descida da árvore), API estilo syscall.

Descrição: Sistema de arquivos completo dentro de um único arquivo binário: superbloco, bitmap de blocos livres, inodes com ponteiros diretos e indiretos, índice de caminhos na B+ e shell interativo (ls, cp, mv, tree, df).

Teste de fogo (anti-atalho): Injetor de falhas (CRASH_AFTER=k aborta na k-ésima escrita); o fsck não pode encontrar corrupção para nenhum k. Operação rename atômica entre diretórios cruzando vários nós da B+ em uma transação.

Tema 2 — Índice de Banco de Dados com Árvore B (registros em disco)

Árvore utilizada: Árvore B (clássica, dados nos nós internos).

Estrutura de dados (núcleo de ED): Árvore B de ordem alta calculada em função do tamanho do bloco; chave primária inteira apontando para registros de tamanho variável; busca, inserção, remoção com redistribuição/fusão e percurso ordenado, tudo em disco.

Mecanismo de SO (núcleo de SO): Buffer cache de páginas com política de substituição (LRU + segunda política comparada), pin/unpin de páginas, persistência via fseek/fread/fwrite, durabilidade com fsync nos commits.

Descrição: Mini-SGBD de tabela única (ex.: matrícula para registro de aluno) com linguagem de comandos insert/select/delete/range e EXPLAIN mostrando quantas páginas cada consulta tocou.

Teste de fogo (anti-atalho): Carga de 10^6 chaves; medir o número médio de acessos a página por busca para 3 tamanhos de bloco e confrontar com a previsão $O(\log_m n)$; comparar a árvore B contra uma B+ (segunda variante) em consultas por intervalo.

Tema 3 — Tabela de Páginas Multinível com Trie de Tradução de Endereços

Árvore utilizada: Trie (radix tree) de tradução de endereços, estilo tabela de páginas x86-64 de 4 níveis.

Estrutura de dados (núcleo de ED): Trie de prefixos do endereço virtual com criação preguiçosa de subníveis, compartilhamento de subárvores entre processos (fork copy-on-write) e liberação de subárvores vazias.

Mecanismo de SO (núcleo de SO): Tratamento de page fault, algoritmos de substituição (FIFO, LRU, Clock, Aging) intercambiáveis, TLB como cache da trie, swap em arquivo binário.

Descrição: Simulador de memória virtual dirigido por traces reais de acesso (gerados com Valgrind/Lackey). Cada nível da trie é percorrido como um caminharmento real de page walk; a TLB encurta o caminho.

Teste de fogo (anti-atalho): Reproduzir a anomalia de Belady com FIFO e mostrar que o LRU não a sofre; medir o hit ratio da TLB e quantos níveis da trie são percorridos por acesso, sob acesso sequencial vs. aleatório.

Tema 4 — Escalonador CFS com Árvore Rubro-Negra por vruntime

Árvore utilizada: Árvore rubro-negra ordenada por tempo virtual de execução (vruntime).

Estrutura de dados (núcleo de ED): Árvore rubro-negra com inserção, remoção e busca do mínimo em $O(\log n)$; a tarefa de menor vruntime fica sempre na folha mais à esquerda (cacheada); reinserção da tarefa após cada fatia de CPU.

Mecanismo de SO (núcleo de SO): Simulação de preempção por fatia de tempo, pesos por nice, contabilização de vruntime ponderado, comparação experimental com Round-Robin e MLFQ.

Descrição: Reimplementação didática do Completely Fair Scheduler do Linux, orientada a eventos discretos, com cargas sintéticas (interativas, batch, mistas) descritas em arquivo. Bônus: escalonar threads reais com SIGALRM + ucontext.

Teste de fogo (anti-atalho): Demonstrar empiricamente a justiça do CFS (CPU proporcional ao peso) e o efeito da árvore rubro-negra na latência de escalonamento; comparar tempo de resposta de tarefas interativas contra RR e MLFQ sob a mesma carga.

Tema 5 — Key-Value Store com LSM-Tree (árvore de níveis em disco)

Árvore utilizada: LSM-Tree: árvore de níveis de SSTables em disco, alimentada por uma árvore balanceada em memória.

Estrutura de dados (núcleo de ED): Memtable como árvore balanceada ordenada (AVL ou treap) que, ao encher, é despejada como SSTable imutável; níveis de SSTables com índice esparsa e Bloom filter; merge k-vias na compactação entre níveis.

Mecanismo de SO (núcleo de SO): Write-ahead log para durabilidade, thread de compactação em background, leitores concorrentes sem bloquear escritores (snapshot por geração), fsync nos pontos corretos.

Descrição: Banco chave-valor estilo LevelDB em miniatura: put, get, delete (tombstones), scan por intervalo. A compactação funde níveis removendo versões obsoletas e tombstones.

Teste de fogo (anti-atalho): kill -9 durante escrita intensa; ao reabrir, o replay do WAL não pode perder escrita confirmada. Medir amplificação de escrita/leitura e o efeito do Bloom filter nas leituras negativas.

Tema 6 — Roteador IP por Longest Prefix Match (Trie de Patricia)

Árvore utilizada: Trie de Patricia / árvore radix comprimida de prefixos IP.

Estrutura de dados (núcleo de ED): Árvore radix comprimida que armazena prefixos CIDR; busca por longest prefix match, inserção e remoção de rotas com recompressão de caminhos; variante multibit (stride) como otimização comparativa.

Mecanismo de SO (núcleo de SO): Plano de dados concorrente: múltiplas threads consultando a tabela enquanto uma thread de controle atualiza rotas sem travar as consultas (RCU simplificado / leitura sem lock com versionamento).

Descrição: Simulador de tabela de roteamento que carrega tabelas BGP reais (datasets públicos) e encaminha um fluxo de pacotes sintéticos, reportando a interface de saída de cada um.

Teste de fogo (anti-atalho): Carregar uma tabela com 10^5 prefixos reais e sustentar consultas concorrentes de 8 threads enquanto rotas são inseridas/removidas; provar com TSan a ausência de races e medir a vazão de lookups por segundo vs. a versão protegida por mutex.

Tema 7 — Deduplicador de Arquivos com Árvore de Merkle

Árvore utilizada: Árvore de Merkle (árvore de hashes) por arquivo.

Estrutura de dados (núcleo de ED): Divisão de arquivos em chunks por rolling hash de Rabin (content-defined chunking); árvore de Merkle que resume o conteúdo de cada arquivo; índice persistente de fingerprints para o chunk único armazenado.

Mecanismo de SO (núcleo de SO): Thread pool com fila de trabalho para varrer diretórios reais e calcular hashes em paralelo; I/O com mmap vs. read (comparação); armazenamento próprio dos chunks deduplicados.

Descrição: Ferramenta de linha de comando que ingere diretórios reais em um repositório deduplicado e restaura qualquer arquivo bit a bit; a árvore de Merkle permite detectar rapidamente quais subárvores mudaram entre duas versões.

Teste de fogo (anti-atalho): Dataset com arquivos grandes parcialmente sobrepostos; demonstrar taxa de deduplicação superior à de blocos fixos, usar a comparação de árvores de Merkle para fazer um diff eficiente, e mostrar escalabilidade quase linear até N threads.

Tema 8 — Backup Incremental com Árvore Persistente Copy-on-Write

Árvore utilizada: Árvore de busca persistente (imutável) com versionamento copy-on-write.

Estrutura de dados (núcleo de ED): Árvore balanceada persistente serializada em arquivo binário: cada snapshot é uma raiz que compartilha estruturalmente as subárvores não modificadas; refcount por nó para coleta de nós órfãos.

Mecanismo de SO (núcleo de SO): Copy-on-write em nível de bloco, criação de snapshot como transação atômica (troca da raiz por último, com fsync ordenado), coletor de nós órfãos ao apagar snapshots, agendamento por timers POSIX.

Descrição: Ferramenta que mantém o histórico de um diretório em um repositório binário: snapshot, list, diff A B, restore, prune. Snapshots compartilham fisicamente tudo que não mudou.

Teste de fogo (anti-atalho): 100 snapshots de um diretório com mudanças pequenas devem ocupar pouco mais que o original + deltas; prune de 50 snapshots intermediários não pode quebrar nenhum dos restantes (verificação de integridade completa após a coleta).

Tema 9 — Escalonador de Disco com Árvore Balanceada de Cilindros

Árvore utilizada: Árvore balanceada (AVL ou rubro-negra) ordenada por número de cilindro.

Estrutura de dados (núcleo de ED): Árvore que mantém as requisições pendentes ordenadas por cilindro, permitindo achar em $O(\log n)$ a próxima requisição na direção do braço (sucessor/antecessor) para implementar SCAN, C-SCAN e LOOK; árvore auxiliar por deadline para a variante EDF.

Mecanismo de SO (núcleo de SO): Simulador de disco rotacional (seek proporcional à distância, latência rotacional, transferência) recebendo requisições de múltiplas threads concorrentes; coalescência de requisições adjacentes; aging contra starvation.

Descrição: Comparativo FCFS x SSTF x SCAN x C-LOOK x deadline scheduler (inspirado no mq-deadline do Linux), com cargas sintéticas (sequencial, aleatória, zipfiana) e visualização do movimento da cabeça (saída para gnuplot).

Teste de fogo (anti-atalho): Demonstrar starvation de uma requisição periférica sob SSTF com carga concentrada no centro e provar que o aging limita a espera máxima; confrontar throughput x latência máxima entre as políticas, todas usando a mesma árvore de cilindros.

Tema 10 — Mini Message Broker com Trie de Tópicos (mini-Kafka)

Árvore utilizada: Trie de tópicos hierárquicos com suporte a curingas.

Estrutura de dados (núcleo de ED): Trie cujos nós são componentes de tópico (sensores/+/temperatura), com casamento de curingas (+ e #) durante a publicação; cada folha referencia um log segmentado append-only com índice esparsa (offset para posição no arquivo).

Mecanismo de SO (núcleo de SO): Servidor concorrente com sockets Unix/TCP e thread (ou pool) por conexão, consumidores com offset persistente independente, retenção por descarte de segmentos antigos, flush configurável (N mensagens / T ms).

Descrição: Broker publish-subscribe com produtores e consumidores como programas separados; mensagens sobrevivem ao reinício do broker; um consumidor que ficou offline retoma exatamente de onde parou.

Teste de fogo (anti-atalho): Matar o broker durante publicação intensa — nenhuma mensagem confirmada (com ack) pode sumir nem ser entregue em duplicidade após o restart; medir vazão com 4 produtores e 8 consumidores em 3 políticas de flush, com assinaturas por curinga ativas.

Tema 11 — Compressor de Arquivos com Árvore de Huffman

Árvore utilizada: Árvore de Huffman (construída a partir de um heap binário).

Estrutura de dados (núcleo de ED): Construção da árvore de codificação ótima de Huffman via heap binário (fila de prioridade); geração da tabela de códigos por travessia da árvore; serialização compacta da árvore no cabeçalho do arquivo; decodificação descendo a árvore bit a bit.

Mecanismo de SO (núcleo de SO): Pipeline de threads (leitor → N codificadores → escritor reordenador) com filas limitadas e condvars, compressão por blocos independentes para paralelismo, verificação de integridade por CRC32 por bloco.

Descrição: Utilitário czip/cunzip que comprime arquivos grandes em blocos, mantém a ordem de saída apesar do término fora de ordem dos trabalhadores e descomprime corretamente reconstruindo a árvore do cabeçalho.

Teste de fogo (anti-atalho): Arquivo de 1 GB; demonstrar speedup quase linear até o número de núcleos e identificar quando o estágio de escrita serializa o pipeline; um bloco corrompido manualmente deve ser detectado sem impedir a descompressão dos demais.

Tema 12 — Cache de Páginas com Árvore Splay (localidade temporal)

Árvore utilizada: Árvore splay (autoajustável) indexando páginas em cache.

Estrutura de dados (núcleo de ED): Árvore splay em que cada acesso a uma página a leva para a raiz, fazendo páginas quentes ficarem perto do topo; remoção das páginas frias (folhas profundas) quando o cache enche; comparação com uma implementação LRU clássica (hash + lista).

Mecanismo de SO (núcleo de SO): Cache de blocos de um arquivo grande lido por múltiplas threads, write-back de páginas sujas, flush no fechamento; medição de hit ratio e de profundidade média de acesso.

Descrição: Camada de page cache para um arquivo de dados acessado com padrões de localidade variados; o objetivo é estudar se o autoajuste da splay aproveita localidade temporal melhor que LRU em certas cargas.

Teste de fogo (anti-atalho): Sob cargas com localidade forte (zipfiana) e sem localidade (uniforme), comparar hit ratio e custo amortizado por acesso entre splay e LRU, explicando teoricamente por que a splay vence ou perde em cada caso; garantir thread-safety sem race no TSan.

Tema 13 — Índice Espacial com Quadtree para Simulação de Colisões

Árvore utilizada: Quadtree (árvore de partição espacial) com subdivisão dinâmica.

Estrutura de dados (núcleo de ED): Quadtree que subdivide o espaço quando um quadrante excede a capacidade e funde quando esvazia; inserção, remoção, consulta por região e consulta dos vizinhos próximos de um ponto; rebalanceamento conforme os objetos se movem.

Mecanismo de SO (núcleo de SO): Simulação paralela: o espaço é dividido entre threads trabalhadoras (uma região da quadtree por thread), com sincronização por barreiras a cada passo de tempo e tratamento de objetos que cruzam fronteiras entre regiões.

Descrição: Simulador de N corpos com detecção de colisões: milhares de partículas se movem em um plano e a quadtree reduz a detecção de colisões de $O(n^2)$ para próximo de $O(n \log n)$. Saída para visualização.

Teste de fogo (anti-atalho): Demonstrar empiricamente o ganho da quadtree sobre a verificação par a par $O(n^2)$ para n crescente; medir o speedup com 1, 2, 4 e 8 threads e analisar o desbalanceamento de carga quando as partículas se concentram em uma região.

Tema 14 — Verificador de Integridade de Disco com Árvore de Hash (dm-verity)

Árvore utilizada: Árvore de hash em blocos (Merkle tree de blocos de disco), estilo dm-verity.

Estrutura de dados (núcleo de ED): Árvore de hash construída sobre todos os blocos de uma imagem: folhas são hashes dos blocos de dados, níveis internos hasham os filhos, até uma raiz única; verificação de um bloco exige apenas o caminho folha-raiz ($O(\log n)$ hashes).

Mecanismo de SO (núcleo de SO): Camada de leitura verificada sobre um arquivo de imagem somente-leitura: cada leitura de bloco valida o caminho na árvore de hash antes de entregar os dados; cache dos nós de hash já verificados para reduzir recomputação.

Descrição: Ferramenta que constrói a árvore de hash de uma imagem, persiste a árvore e oferece uma API de leitura que detecta qualquer adulteração de bloco, identificando exatamente qual bloco foi corrompido.

Teste de fogo (anti-atalho): Corromper manualmente um único byte em um bloco aleatório da imagem; a leitura desse bloco deve falhar apontando o bloco exato, enquanto todos os demais continuam legíveis; medir o overhead da verificação com e sem cache dos nós de hash.

Tema 15 — Coletor de Lixo Geracional com Árvore de Intervalos de Memória

Árvore utilizada: Árvore de intervalos (interval tree, balanceada) sobre as regiões de memória alocadas.

Estrutura de dados (núcleo de ED): Árvore de intervalos que mapeia endereços para os objetos alocados, permitindo, durante a varredura conservadora, decidir em $O(\log n)$ se um valor encontrado na pilha aponta para dentro de algum objeto vivo; conjuntos de raízes e fila de marcação.

Mecanismo de SO (núcleo de SO): Heap gerenciado via mmap, varredura conservadora de pilha e registradores (setjmp / inspeção do stack pointer), barreira de escrita com mprotect na variante geracional (2 gerações), medição de pausas com clock monotônico.

Descrição: Biblioteca gc_malloc() sem free: a coleta dispara por pressão de memória ou explicitamente. Programas-cobaia obrigatórios: listas, árvores e grafos com ciclos (que contagem de referência não coletaria).

Teste de fogo (anti-atalho): Programa que cria 10^7 objetos com ciclos e referências cruzadas — o GC não pode coletar objeto vivo (verificação por canários) nem vazar os mortos (uso de memória estabiliza); gráfico de tempo de pausa x tamanho do heap comparando mark-sweep puro contra a variante geracional.